

COGNOME:

NOME:

MATRICOLA:

Prova Parziale di Algoritmi e Strutture Dati, 21 Aprile 2026

Esercizio 1 [5 punti]

Quale delle seguenti sequenze di funzioni è tale che ogni funzione è O della successiva?

(A) $n \log n$, $\sum_{i=n/2}^n \log i$, n^{2026} , $n^{(\log n)^{2026}}$, $2^{\sqrt{n} \log n}$

(B) $(\log n^{\sqrt{n}})^2$, $n \log n!$, $n^{2.001}$, $\frac{(\log n)^{2026} + n^3 + n}{n^{3/2}}$, $\binom{n}{3}$

(C) $n + (\log n)^{2026}$, $\frac{n^2}{\log n}$, $2^{n/2}$, $n^{\log n}$, $\sum_{i=0}^n 2^i$

~~(D)~~ Nessuna delle altre

Esercizio 2 [5 punti]

Si consideri il seguente algoritmo, in cui l'input è un intero positivo n potenza di 2. Qual è il numero di chiamate alla funzione f in $\text{ALGO}(n)$?

Algorithm $\text{ALGO}(n)$

```

1: if  $n > 1$  then
2:    $i = 1$ 
3:   while  $i \leq n$  do
4:     for  $j = 1, \dots, i$  do
5:        $f()$ 
6:     end for
7:      $i = 2 \cdot i$ 
8:   end while
9:    $i = n^2$ 
10:  while  $i \geq 1$  do
11:     $f()$ 
12:     $i = i/2$ 
13:  end while
14:   $\text{ALGO}(n/2)$ 
15: end if

```

$$T(n) = T(n/2) + \Theta(n)$$

$$a=1 \quad b=2 \quad n^{\log a} = n^0 = 1 < n$$

$$T(n) = \Theta(n)$$

(A) $\Theta(n^2)$

(B) $\Theta(\log n)$

~~(C)~~ $\Theta(n)$

(D) Nessuna delle altre

Esercizio 3 [5 punti]

Vogliamo verificare se gli elementi di un array di lunghezza n sono tutti distinti. Quale delle seguenti affermazioni è vera?

(A) È possibile utilizzare una tavola hash per risolvere il problema in $O(n)$ nel caso peggiore.

~~(B)~~ È possibile utilizzare MERGE-SORT per risolvere il problema in $O(n \log n)$ nel caso peggiore.
 $n \rightarrow \text{for}(i: \text{array})$

(C) L'algoritmo brute force che confronta ciascuna coppia di elementi richiede $\omega(n^2)$ confronti.

(D) Nessuna delle altre.

Esercizio 4 [5 punti]

Quale delle seguenti affermazioni è falsa?

- (A) È possibile ordinare un array di n interi compresi fra $-n^3$ e n^3 in $O(n)$. *→ radix sort*
→ scomposizione e confronto con il pivot
- (B) La fase Divide di QUICKSORT è eseguita in $O(n)$ nel caso peggiore per input di dimensione n .
- (C) Sia A una sequenza di n elementi, ciascuno colorato di rosso o di blu, rappresentata tramite array. È possibile ordinare A in loco in modo che ciascun elemento blu preceda ciascun elemento rosso e utilizzando $O(n)$ confronti. *→ 2 puntatori: uno all'inizio e l'altro alla fine e scambio i rossi a sx con quelli blu a dx*
- ~~(D)~~ Sia A un array di n elementi partizionati in $\sqrt{n}$ subarray, ciascuno di $\sqrt{n}$ elementi. Gli elementi di ciascun subarray sono tutti più piccoli degli elementi del subarray successivo e più grandi degli elementi del subarray precedente. Un possibile tale array A con $n = 9$ è $\langle 2, 1, 3, 6, 5, 4, 10, 20, 30 \rangle$. È possibile ordinare A utilizzando $O(n)$ confronti.

Esercizio 5 [5 punti]

Vogliamo ordinare un array A di n interi in senso non-decrescente. Sia FANCY-SORT un algoritmo basato su confronti che, dato in input un tale A , effettua una serie di swap tra elementi adiacenti fino ad ordinare A . Quale delle seguenti affermazioni è vera?

- (A) Il numero di swap effettuati da FANCY-SORT nel caso peggiore è $O(n \log n)$. *α n² scambi*
- ~~(B)~~ Se A è formato da n interi distinti, allora non esiste un algoritmo che utilizza $o(n^2)$ confronti nel caso peggiore per determinare un lower bound sul numero di swap effettuati da FANCY-SORT con input A .
- (C) FANCY-SORT non può essere reso stabile mantenendo asintoticamente il numero di confronti effettuati nel caso peggiore. *α gli algoritmi basati su swap adiacenti: es. insertion sort è stabile.*
- (D) Nessuna delle altre.

SBARRAMENTO - 15 punti sugli esercizi precedenti

Esercizio 6 [10 punti] Dati due insiemi A e B , sia $\eta(A, B) = |A \cap B|/|A|$. Supponiamo che A e B abbiano n elementi ciascuno e siano memorizzati in due array. Vogliamo progettare un algoritmo per calcolare $\eta(A, B)$.

1. Illustrare brevemente l'idea di un algoritmo che risolve il problema utilizzando $O(n^2)$ confronti. [2pt]
2. Fornire lo pseudo codice di tale algoritmo. [3pt]
3. Determinare il tempo di esecuzione dell'algoritmo. [2pt]
4. Dimostrare la correttezza dell'algoritmo. [3pt]

NB: Punteggio pieno solo per algoritmi con $o(n^2)$ confronti.

Esercizio 1 [5 punti]

Quale delle seguenti sequenze di funzioni è tale che ogni funzione è O della successiva?

(A) $n \log n$, $\sum_{i=n/2}^n \log i$, n^{2026} , $n^{(\log n)^{2026}}$, $2^{\sqrt{n} \log n}$

(B) $(\log n^{\sqrt{n}})^2$, $n \log n!$, $n^{2.001}$, $\frac{(\log n)^{2026} + n^3 + n}{n^{3/2}}$, $\binom{n}{3}$

(C) $n + (\log n)^{2026}$, $\frac{n^2}{\log n}$, $2^{n/2}$, $n^{\log n}$, $\sum_{i=0}^n 2^i$

(D) Nessuna delle altre

A. $n \log n \leq \sum_{i=n/2}^n \log i \leq n^{2026} \leq n^{(\log n)^{2026}} > 2^{\sqrt{n} \log n}$

• $\sum_{i=n/2}^n \log i \approx \log n! \approx n \log n$

• $n^{(\log n)^{2026}} = (2^{\log n})^{\log n^{2026}} = 2^{\log n \cdot \log n^{2026}} = 2^{\log n^{2027}}$

B. $(\log n^{\sqrt{n}})^2 \leq n \log n! \leq n^{2.001} > \frac{(\log n)^{2026} + n^3 + n}{n^{3/2}}$

• $(\log n^{\sqrt{n}})^2 = (\sqrt{n} \log n)^2 = n \log n \log n$

• $n \log n! = n \cdot n \log n = n^2 \log n$

• $\frac{(\log n)^{2026} + n^3 + n}{n^{3/2}} \sim \frac{n^3}{n^{3/2}} = n^{3-3/2} = n^{3/2} = n^{1.5}$

C. $n + (\log n)^{2026} \leq \frac{n^2}{\log n} \leq 2^{n/2} > n^{\log n}$

• $\frac{n^2}{\log n} = n^2 \cdot \frac{1}{\log n}$

• $2^{n/2} = 2^{1/2 n} = \sqrt{2}^n$

• $n^{\log n} = (2^{\log n})^{\log n} = 2^{\log n \log n}$

Esercizio 6 [10 punti] Dati due insiemi A e B , sia $\eta(A, B) = |A \cap B|/|A|$. Supponiamo che A e B abbiano n elementi ciascuno e siano memorizzati in due array. Vogliamo progettare un algoritmo per calcolare $\eta(A, B)$.

1. Illustrare brevemente l'idea di un algoritmo che risolve il problema utilizzando $O(n^2)$ confronti. [2pt]
2. Fornire lo pseudo codice di tale algoritmo. [3pt]
3. Determinare il tempo di esecuzione dell'algoritmo. [2pt]
4. Dimostrare la correttezza dell'algoritmo. [3pt]

NB: Punteggio pieno solo per algoritmi con $o(n^2)$ confronti.

```

1. ordiniamo entrambi gli array A e B in ordine crescente usando il merge sort
   . Scansione con 2 puntatori: se  $A[i] == B[j]$   $\rightarrow$  un elemento comune.
                                se  $A[i] < B[j]$   $\rightarrow$  l'elemento in A è piccolo,
                                avanziamo solo j.
                                se  $A[i] > B[j]$   $\rightarrow$  avanziamo solo i.

2. ALGOCA, B, n)
   MERGE-SORT(A)  $\rightarrow \theta(n \log n)$ 
   MERGE-SORT(B)  $\rightarrow \theta(n \log n)$ 

   i = 1
   j = 1
   comuni = 0
   WHILE i  $\leq$  n & j  $\leq$  n do  $\rightarrow \theta(n)$ 
       IF  $A[i] == B[j]$   $\rightarrow \theta(n \log n)$ 
           comuni++
           i++
           j++
       elseif  $A[i] < B[j]$ 
           i++
       else
           j++
   return comuni/n

T(n) =  $\theta(n \log n) \cdot 2 + \theta(n) = \theta(n \log n)$ 

```

4. Supponiamo per assurdo che l'algoritmo sia errato e che termini la scansione lasciando fuori un elemento comune x senza averlo incrementato in comuni.

• Siano i e j le posizioni in cui si trova l'elemento comune $x \rightarrow A[i] = B[j] = x$

• Per non essere stato contato, l'algoritmo deve aver saltato una di queste due posizioni prima che gli indici si incontrassero. Supponiamo che abbia avanzato l'indice i oltre la posizione del valore x quando l'altro indice si trova ancora indietro ($B[j']$, con $j' < j$)

• L'indice i viene incrementato senza contare solo se: $A[i] < B[j']$

Ma poiché l'array B è ordinato in modo crescente e $j' < j$, sappiamo che $B[j'] \leq B[j]$.

Sostituendo i valori avremmo: $x < B[j'] \leq x \implies x < x$
 una contraddizione.